

Laghlali Noah
Rochedreux Hugo
Klingler Emma

RA-IL2

THE SILENT HUNT



2025-2026

Tuteur : Monsieur Olivier Perrin

Sommaire

Sommaire	1
1. Introduction	3
2. Analyse du projet	4
2.1 Évolution par itérations	5
3. Réalisation	6
3.1 Architecture logicielle	6
3.1.1 Description	6
3.1.2 Pourquoi cette structure a été choisie	8
3.2 Arbre de comportement (BT)	9
3.2.1 Sélecteur de poids	10
3.2.1.1 Sélection du noeud	11
3.2.1.2 Exécution du noeud	11
3.2.2 Utility System	12
3.2.2.1 Appliquer l'Utility au chasseur	12
3.2.2.2 Formaliser le chasseur	14
4. Tests de validation	16
5. Difficultés rencontrées	17
6. Itération du projet	18
Itération 1 :	18
Itération 2	21
Itération 3	23
Objectifs	23
Avant l'itération	23
Déroulement	24
Itération 4	27
Objectifs de l'itération	27
Ajout de mécaniques buissons et terrier	27
Caméra spectateur	27
Utility actions	28
Itération 5	31
Objectifs de l'itération	31
État de la version	31
Finalisation des calculs d'Utility	31
Bot lapin	35
Difficultés rencontrées	36
Itération 6	37
Objectifs de l'itération	37
État de la version	37

Correction des bugs du lapin	37
Ajustement des paramètres du chasseur	37
Préparation des livrables finaux	37
7. Répartition du travail	38
Planning de déroulement du projet durant les 6 itérations	39
Conclusion	40
Annexe	41
Lancer le jeu	41
Installation du jeu	43

1. Introduction

Ce rapport présente l'état de notre projet tutoré The Silent Hunt finalisé. Ce projet a pour but de suivre les objectifs, la mécanique du jeu ainsi que les choix techniques définis par l'étude préalable effectuée en début de semestre 5.

The Silent Hunt est un jeu de survie développé sur la plateforme de jeu Roblox à l'aide du langage Lua. Le joueur incarne un lapin évoluant dans une forêt, poursuivi par un chasseur contrôlé par une intelligence artificielle. Le but du jeu est de survivre le plus longtemps possible en gérant plusieurs ressources essentielles comme la vie, la faim et le stress, tout en utilisant l'environnement pour se cacher et fuir.

L'un des éléments principaux du projet est l'intelligence artificielle du chasseur. Celle-ci repose sur un Behaviour Tree, utilisé comme structure de décision principale. Celui-ci est complété par une approche de type Utility AI, permettant d'ajuster dynamiquement les priorités des actions selon le contexte de jeu, et ainsi d'adapter le comportement du chasseur au joueur.

Ce document présente l'analyse du projet, le déroulement des différentes itérations des deux semestres, les choix techniques réalisés et la répartition du travail entre les membres du groupe.

2. Analyse du projet

Le lapin constitue le cœur du gameplay :

- déplacements (marcher, courir, sauter)
- actions principales : manger, se cacher
- caméra en première personne pour l'immersion
- gestion de la faim, de la santé, et de la mort
- HUD synchronisé entre client et serveur

Le chasseur, de son côté, utilise un Behaviour Tree lui permettant :

- de patrouiller, enquêter et poursuivre
- de poser des pièges, recharger, chercher des munitions
- d'adopter un comportement crédible et réactif

L'environnement inclut une carte de test avec une petite forêt, des zones, et des terriers sont prévus, servant à la fois de cachettes et de téléportation.

Le jeu intègre un mode multijoueur, avec synchronisation client/serveur, des objets interactifs (carottes) et un lobby permettant de créer ou rejoindre une partie, gérer les joueurs et configurer le nombre de lapins IA. Une IA de lapin simple a également été ajoutée pour plus tard entraîner le chasseur.

2.1 Évolution par itérations

Itération 1 : Prototype solo

Les objectifs initiaux portaient sur le déplacement, la survie, et une IA simple du chasseur (FSM).

Itération 2 : Interaction et premières mécaniques avancées

L'itération devait introduire le système multijoueur, la détection de bruit et surtout le début de BT pour le chasseur.

Itération 3 : IA avancée (partiellement atteinte)

L'objectif était d'ajouter le système de piège, de terrier et d'améliorer le BT du chasseur.

Itération 4 : Fonctionnalités lapin et utility

Les objectifs étaient d'ajouter des fonctionnalités pour le joueur, comme le terrier et le buisson. De plus, commencer les calculs d'utility pour le chasseurs.

Itération 5 : Expériences et interfaces utilisateur et bot lapin

L'itération avait pour but de finir les utility du chasseurs ainsi que d'améliorer le côté graphique du client, ajout d'interface, minimap et l'ajout de bot lapin.

Itération 6 : Ajustement AI

L'objectif était de changer les paramètres du chasseur afin de rendre la partie la plus pertinente.

3. Réalisation

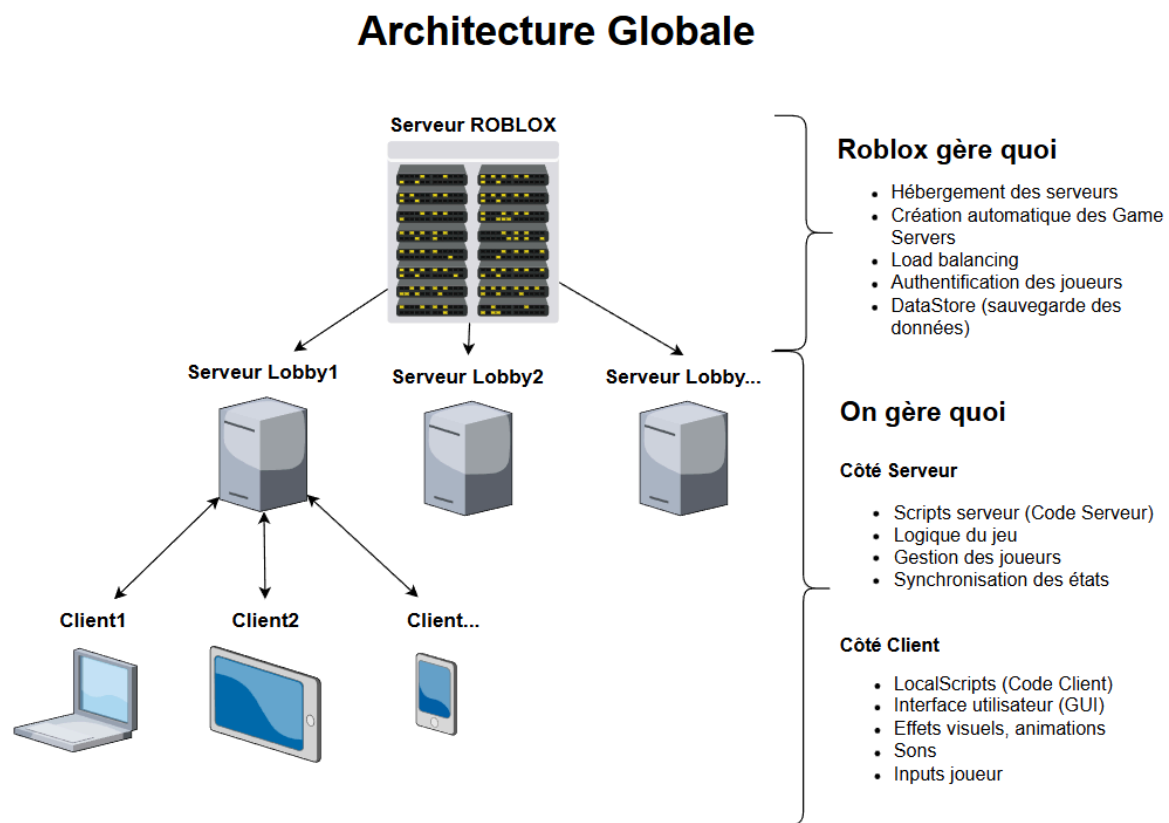
3.1 Architecture logicielle

3.1.1 Description

L'architecture de The Silent Hunt repose sur trois outils principaux : **Roblox Studio**, **Rojo** (VS Code) et **GitHub**.

Roblox Studio est un environnement de développement (IDE) spécialisé dans la conception de jeux vidéo.

Voici l'architecture de roblox pour mieux comprendre.



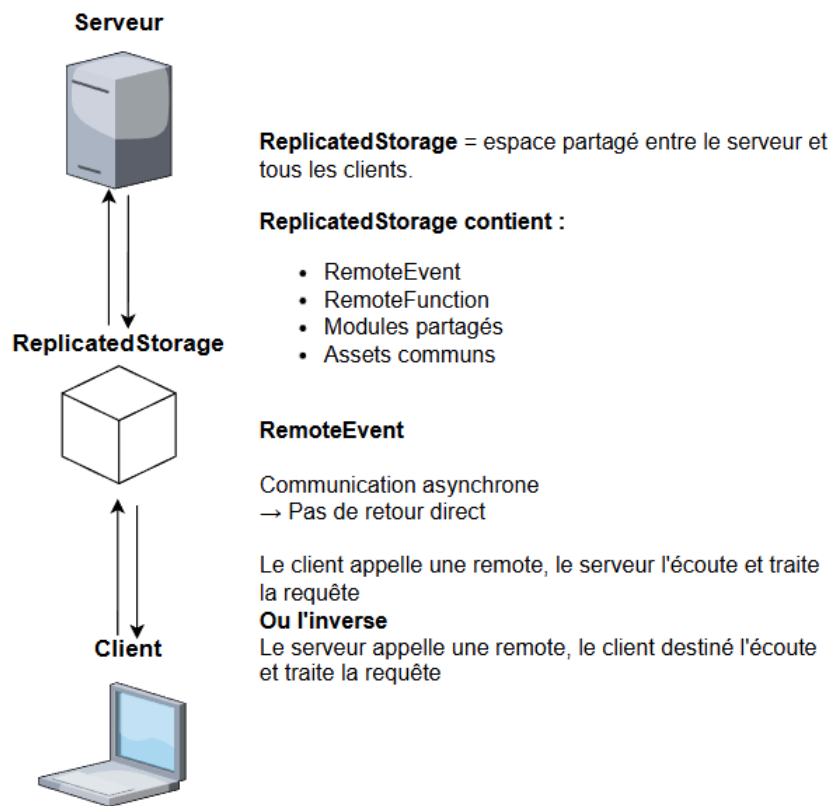
1. Architecture de Roblox

Rojo synchronise automatiquement vers Roblox Studio, ce qui évite les scripts dispersés et difficiles à retrouver dans Roblox Studio.

GitHub sert au versioning, au travail en groupe et à la sauvegarde des différentes itérations du projet.

Une partie importante de l'architecture est la communication entre serveur et clients. Pour mieux comprendre, voici un schéma des échanges :

Communication



2. Système de communication entre client et serveur

En résumé, **ReplicatedStorage** est un espace partagé. Il est composé de **RemoteEvents**, ce qui permet la communication entre serveur et clients.

Dans Roblox, le serveur est l'instance principale : IA, logique du jeu, état global, etc.

Donc si le serveur s'arrête :

- L'IA du chasseur s'arrête aussi (scripts serveur stoppés).
- Les joueurs sont automatiquement déconnectés (Roblox ferme la session).
- L'état du jeu est perdu.
- Aucun script client ne peut continuer la partie, car ils dépendent des RemoteEvents/RemoteFunctions du serveur.

3.1.2 Pourquoi cette structure a été choisie

Séparation claire des responsabilités :

- Serveur : IA, règles du jeu, systèmes critiques.
- Client : interface, contrôles, caméra.
- ReplicatedStorage : modules partagés (RemoteEvent).

Cette organisation suit les standards Roblox, donc moins de conflits, meilleure lisibilité, comportement multijoueur cohérent.

Rojo + VS Code

- Édition du code propre, structurée en dossiers.
- Synchronisation automatique, donc aussi rapide que d'éditer dans Roblox Studio.

GitHub

- Versioning, historique clair.
- Travail en équipe facile.

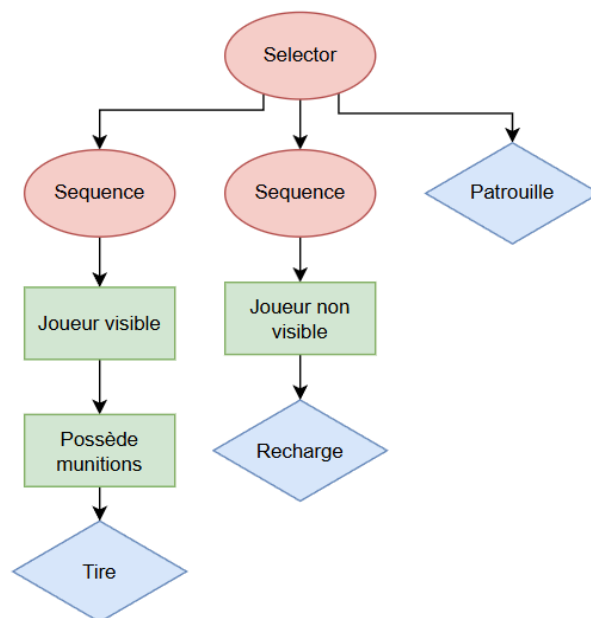
Nous avons décidé de choisir Roblox Studio car c'est un logiciel assez simple à prendre en main comparé à des logiciels comme Unity ou Unreal Engine, plus puissant mais plus compliqué.

3.2 Arbre de comportement (BT)

Un arbre de comportement ou Behaviour Tree (BT), c'est une structure modulaire, un arbre qui fonctionne avec des nœuds de type **sélection**, **action** ou **condition**.

Il s'exécute à chaque frame et va exécuter un nœud de type sélection, qui va choisir un nœud à exécuter, et ainsi de suite jusqu'à exécuter un nœud action qui retourne soit **SUCCESS**, **FAILURE** ou **RUNNING**.

Si on prend un exemple simple pour le chasseur :



3. BT simple du chasseur

Donc à chaque exécution notre arbre va être déroulé depuis la racine. Notre arbre va être parcouru de gauche à droite jusqu'à ce qu'un nœud soit valide. Si on prend notre *Image* ci-dessus, notre arbre va regarder si le joueur est visible, si le chasseur possède des munitions et si oui, alors il va tirer. Sinon il passe à la branche suivante jusqu'à ce qu'une branche renvoie **RUNNING** ou **SUCCESS**.

On a donc une structure très modulaire et assez simple, on pourrait dire parfaite pour notre chasseur.

Dans un arbre de comportement les priorités sont statiques. Elles sont directement inscrites dans notre arbre. Cela le rend très lisible mais en pratique c'est très limitant.

Or dans notre jeu, une action peut avoir une priorité différente en fonction du contexte. Cela revient donc souvent à dupliquer une section de notre arbre en changeant les conditions, ce n'est pas pratique et ne résout pas le problème.

En réalité, une décision n'est jamais binaire, si on prend notre chasseur en exemple (*Image 3. BT simple du chasseur*), l'action Recharge ne se fait que lorsque le joueur n'est pas visible. Or si le chasseur ne possède plus de balle dans son chargeur mais voit un joueur, il ne pourra jamais recharger ce qui pose un problème.

La solution à tous ces problèmes est l'intégration de l'utility.

(Plus de détails 10.2, 10.3 [Documentation](#))

3.2.1 Sélecteur de poids

L'objectif d'un sélecteur de poids (Weighted Selector) est de sélectionner une action parmi ses nœuds enfants en fonction de leur poids.

Un sélecteur de poids possède une liste d'enfants composée de:

- Un nœud Action cf(ActionChasseur)
- Une variable **bloquer** qui permet de bloquer l'action si jamais elle est en cours. Par exemple : une fois que le nœud Recharger est en cours, le sélecteur ne changera pas de nœud jusqu'à la fin de l'action.
- Une variable **priorité** qui permet de gérer la priorité de chaque action. Par exemple, si le nœud **Poursuivre** est de priorité 2, le nœud Explorer de priorité 1 et qu'il explore, il peut être interrompu par la poursuite, mais inversement non.
- Une variable **poids** qui est l'utility de l'action

Pour résumer notre Behaviour tree boucle à chaque frame du jeu, et va calculer les poids de chaque action ensuite il va exécuter le sélecteur de poids qui va donc sélectionner l'action à choisir.

3.2.1.1 Sélection du noeud

Le sélecteur va regarder quel nœud enfant a le **poids** le plus élevé ainsi que l'ensemble des **poids** de tous ses enfants.

Par exemple, **60%** du temps le nœud avec le poids le plus fort sera choisi, sinon **40%** du temps un enfant aléatoire en fonction des poids sera choisi, cela permet au chasseur d'être 60% du temps "parfait" et 40% plus humain en choisissant de manière plus aléatoire.

3.2.1.2 Exécution du noeud

Premièrement, le sélecteur vérifie si la variable **bloquer** du nœud en cours est à vrai, si oui, il continue d'exécuter le nœud, sinon, il va sélectionner un nouveau nœud et va comparer sa priorité avec celle du nœud courant. Si le nouveau est plus prioritaire alors il va le remplacer sinon, le nœud courant n'est pas remplacé.

Un système d'hystérésis a été mis en place afin d'éviter des changements d'action trop fréquents. Le sélecteur ne change de comportement que si la nouvelle action possède un poids suffisant par rapport à l'action actuelle (seuil fixé à 85 %). Cela permet d'éviter des alternances constantes entre deux actions de même importance.

Enfin, afin d'augmenter les performances et de réduire les calculs, nous avons ajouté un temps pour la réévaluation du nœud. Le sélecteur change une fois toutes les 0.25 secondes, soit 4 fois par seconde. En dessous de 4 fois par seconde, cela serait inutile car invisible à l'œil nu.

3.2.2 Utility System

Un utility system est une méthode de prise de décision utilisée en IA dans les jeux pour mesurer à quel point une action est pertinente par rapport à un moment donné.

Au lieu de décider si une action est valide ou non comme vu dans la section de l'arbre de comportement, on attribue un score (une utilité) à chaque action possible. On compare ensuite ces scores et on sélectionne l'action correspondante grâce à notre **Weighted Selector**.

L'idée, comme vu précédemment, est qu'il n'y a presque jamais une seule "bonne" décision mais il existe souvent plusieurs options raisonnables.

L'utility system permet donc de mesurer la fiabilité d'une action et de choisir la meilleure.

3.2.2.1 Appliquer l'Utility au chasseur

Pour chaque action possible, on calcule une valeur d'utilité basée sur différents facteurs.

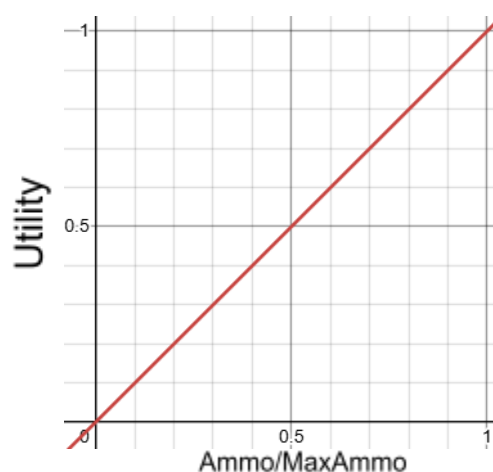
Prenons comme exemple le rechargement (Reload).

On peut premièrement mesurer certains attributs du chasseur :

- **Ammo** : nombre de balles dans le fusil du chasseur
- **MaxAmmo** : nombre de balles maximum dans le fusil du chasseur

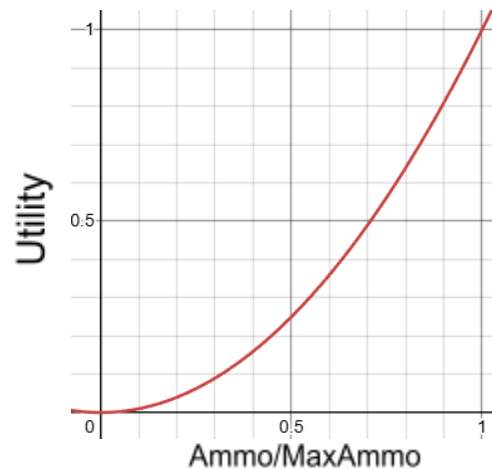
On pourrait alors créer une formule simple :

$$Utility(Recharge) = 1 - (Ammo/MaxAmmo)$$



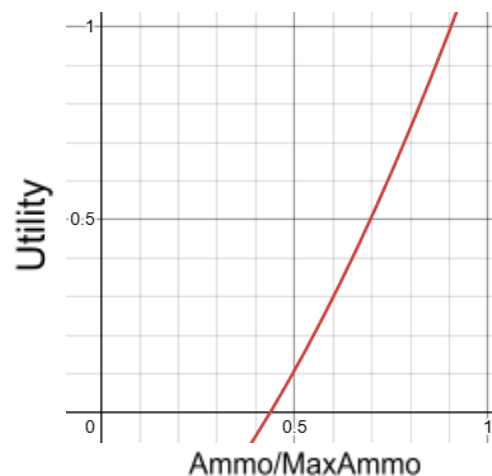
Avec notre formule simple on peut voir un problème. Si notre chasseur possède encore beaucoup de balles dans son chargeur, il y a quand même une chance qu'il recharge. C'est pourquoi on peut modifier l'allure de la courbe en appliquant la fonction puissance, ce qui nous donne une nouvelle formule.

$$Utility(Recharge) = 1 - (pow(Ammo/MaxAmmo))$$



Cette nouvelle courbe est plus intéressante car l'utilité augmente de manière plus douce au début et plus forte à la fin. On peut bien sûr aller encore plus loin en ajoutant quelques ajustements à notre fonction.

$$Utility(Recharge) = 1 - (pow(Ammo/MaxAmmo)+0.4)-0.7$$



Maintenant cette courbe est encore plus intéressante. Lorsque le chargeur du chasseur est rempli l'utilité est à 0 et c'est seulement quand le chargeur est à moitié vide que l'utilité va augmenter de manière exponentielle.

Bien évidemment ce n'est qu'un exemple simple, en effet on pourrait encore améliorer cette fonction. Une façon de faire serait d'ajouter de nouveaux facteurs comme la fatigue du chasseur ou alors la distance avec un joueur cible.

3.2.2.2 Formaliser le chasseur

L'utilité system nous oblige donc à formaliser notre chasseur. La première étape est d'énumérer les différentes actions du chasseur.

- Attaquer
- Tirer
- Recharger
- Ravitailler
- Poursuivre Cible
- Placer Piège
- Patrouiller

Ensuite, il faut énumérer les attributs du chasseur qui seront nos fameux facteurs pour le calcul d'utilité.

- | | |
|--------------------|---|
| - Fatigue | F |
| - Patience | P |
| - Agressivité | A |
| - BallesChargeur | B |
| - BallesInventaire | I |
| - NombrePièges | T |
| - VieLapin | V |
| - DistanceLapin | D |
| - LigneDeVue | L |
| - Position | O |

Les lettres représentent les curseurs dans [Desmos](#) (site web pour représenter le calcul de poids).

Problème : comment comparer les balles du chargeur avec la fatigue du chasseur ou d'autres attributs ?

On ne peut pas comparer des unités différentes directement.

Solution : **normaliser toutes les valeurs entre 0 et 1.**

Par exemple :

- 0 = pas du tout intéressant
- 1 = extrêmement intéressant

Si on prend en exemple **BallesChargeur**, l'attribut représente le nombre de balles dans le chargeur du chasseur. Avec la normalisation, on aurait notre fameux *Ammo/MaxAmmo*, une valeur comprise entre 0 et 1.

(Détails [Documentation](#))

4. Tests de validation

Les tests ont été menés selon deux approches complémentaires :

Des tests unitaires simples ont été réalisés dans VS Code ainsi que des tests en conditions réelles dans Roblox Studio.

Les tests en jeu ont permis de valider les mécaniques principales :

- déplacements du lapin et gestion correcte des collisions
- déclenchement de la mort lorsque la vie atteint 0
- diminution progressive de la faim et restauration avec les carottes
- affichage cohérent des jauges dans le HUD

Tests de l'IA du chasseur :

- transitions correctes entre patrouille, investigation et poursuite
- cohérence de la vision et perte de la cible après un délai
- vérification de la pose de pièges

Tests multijoueur :

- synchronisation des jauges (vie, faim)
- fonctionnement des objets interactifs
- lobby fonctionnel (rejoindre, exclure, lancer une partie)
- comportement du chasseur face à plusieurs cibles

Tests de performance :

- les scripts ne provoquent pas de ralentissements
- l'IA reste stable avec plusieurs joueurs
- le serveur ne rencontre pas d'erreurs majeures

Ces tests ont été répétés à chaque itération afin de valider les nouvelles fonctionnalités et garantir la stabilité du projet.

5. Difficultés rencontrées

Prise en main du code

- Apprentissage du langage Lua
- Compréhension du modèle client/serveur Roblox
- Découverte des modules, RemoteEvents et de Rojo

Organisation de l'équipe

- Absences fréquentes
- Équipe au complet seulement la dernière semaine
- Nécessité de revoir les priorités et de réduire certaines fonctionnalités

Complexité technique de l'IA du chasseur

- Mise en place d'une IA et d'Utility (ajustement des poids)
- Équilibrage du Behaviour Tree pour éviter une IA trop faible, incohérente ou imbattable
- Nombreux tests nécessaires pour stabiliser le comportement

Ce que nous avons appris

- Travailler efficacement avec un nouveau langage
- S'adapter aux imprévus d'équipe et réorganiser un projet en conséquence
- Concevoir une IA avancée
- Tester, ajuster et itérer rapidement pour obtenir un comportement cohérent
- Prioriser les fonctionnalités essentielles quand le temps est limité

6. Itération du projet

Itération 1 :

Objectif de l'itération

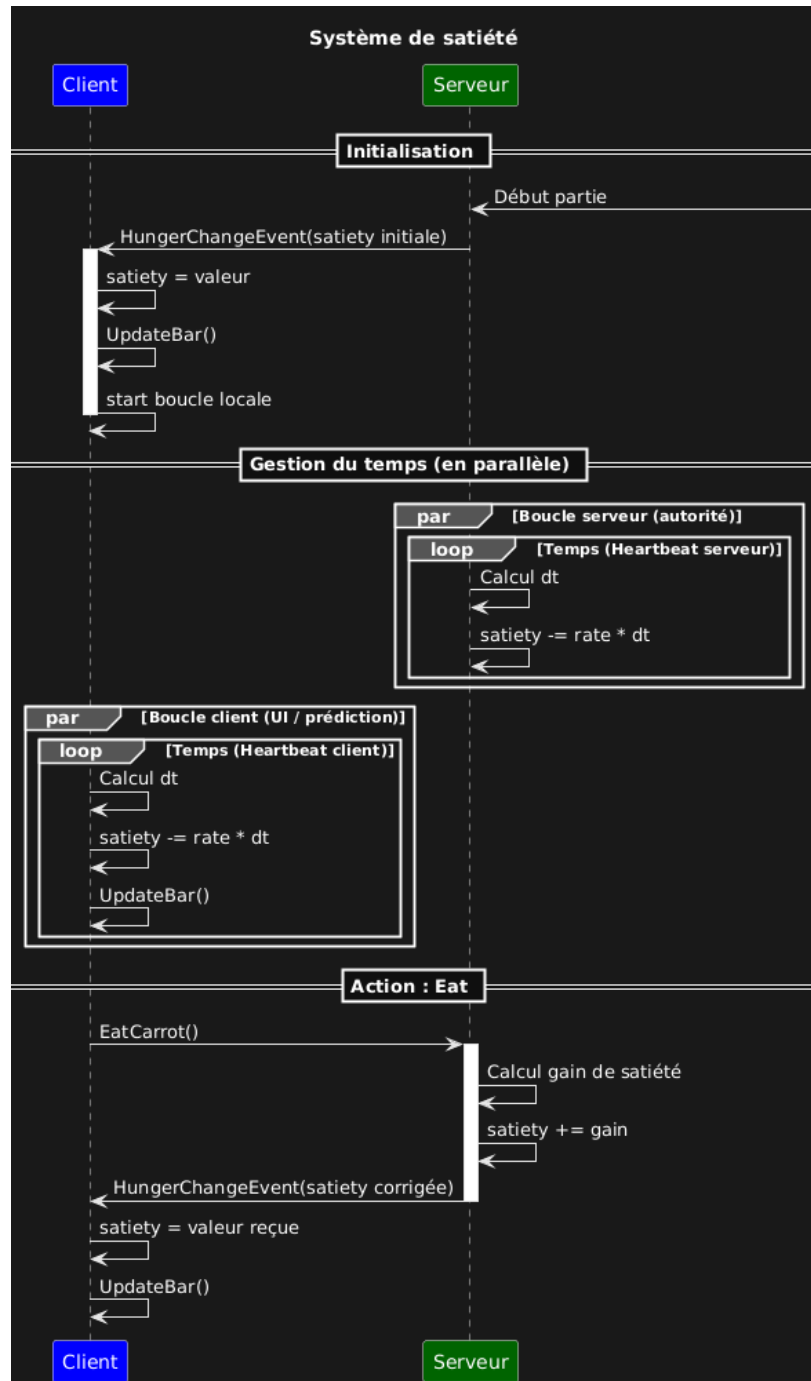
L'objectif était de produire une première version jouable de The Silent Hunt, en posant les bases du gameplay, de l'architecture et des systèmes essentiels.

Prototype fonctionnel

Nous avons suivi le plan établi lors de l'étude préalable et obtenu un prototype intégrant le lapin, le chasseur, la gestion de la vie et de la faim, ainsi qu'une première carte de test.

Système du joueur (le lapin)

- Implémentation d'un lapin jouable et animé
- Déplacements dans l'environnement
- Deux jauges : vie et faim
- La faim diminue progressivement, et la carotte permet de la restaurer
- HUD opérationnel grâce à un système de réplication serveur/client



4. Diagramme de séquence du système de satiété

Système du chasseur

- Introduction du chasseur avec un Behaviour Tree simple
- Patrouille aléatoire
- Détection du lapin et déplacement vers lui
- Attaque de base (un message console)

Carte de test et validation du gameplay

Une petite carte a permis de valider les mécaniques principales.

Un bug notable a été identifié : le lapin peut parfois “voler” dans les airs, un problème prévu pour la prochaine itération.

Compétences

Cette phase a permis à Noah et Emma de se familiariser avec Lua et l’environnement Roblox, en comprenant mieux les interactions entre scripts serveur, scripts client et classes personnalisées.

Itération 2

Objectifs de l'itération

L'objectif principal de l'itération II était de développer une intelligence artificielle du chasseur plus aboutie et jouable. Cette phase avait pour objectif de permettre de passer d'un prototype simple à une version plus proche de la vision définie dans l'étude préalable, avec un chasseur capable de détecter, poursuivre et attaquer le joueur de manière cohérente.

État de la version

À la fin de cette itération, l'IA du chasseur est fonctionnelle et intégrée au gameplay. Le chasseur est désormais capable de :

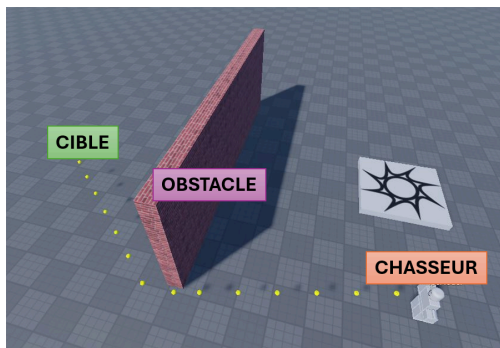
- détecter le lapin grâce à un système de vision basé sur un champ de vision et du raycasting ;
- mémoriser la dernière position connue du lapin ;
- poursuivre le joueur en utilisant le PathfindingService ;
- attaquer au corps à corps et à distance ;
- gérer un chargeur et une réserve de munitions ;
- se recharger et se ravitailler ;
- poser un premier type de piège basé sur la mémoire.

Pathfinding :

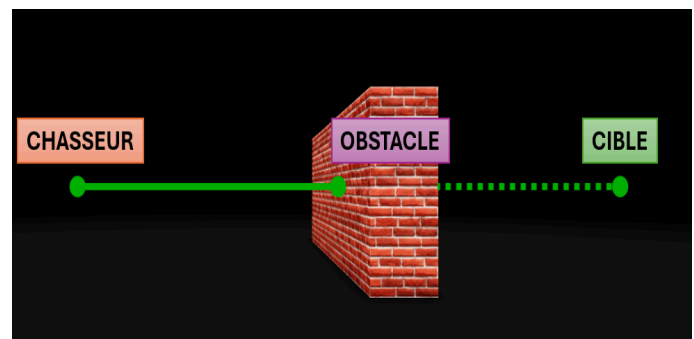
Le pathfinding permet au chasseur de calculer automatiquement un chemin vers le joueur en contournant les obstacles. Ainsi, il ne se déplace pas en ligne droite mais suit un trajet réaliste adapté à l'environnement.

Raycasting :

Le raycasting permet de vérifier la ligne de vue entre le chasseur et le lapin. Si un obstacle bloque le rayon, le tir est empêché, ce qui évite que l'IA tire à travers les murs.

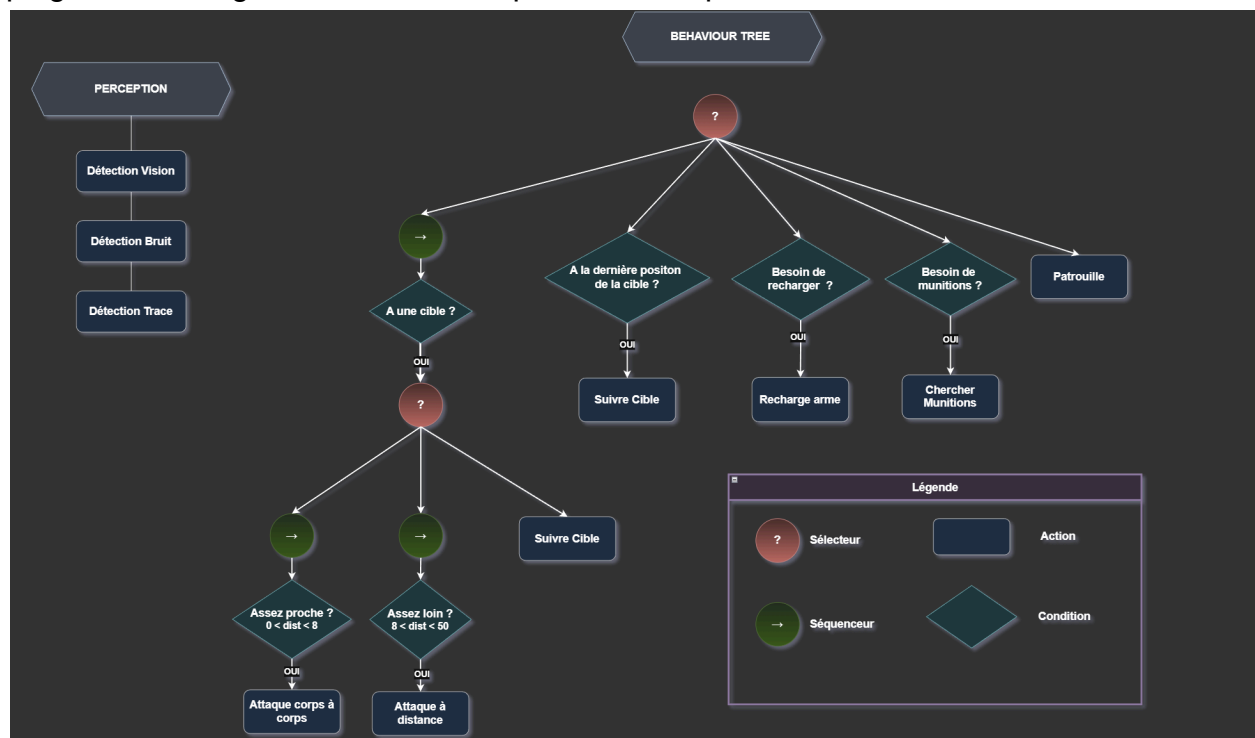


5. Fonctionnement du pathfinding



6. Fonctionnement du raycasting

Le Behaviour Tree a été amélioré afin de mieux gérer les priorités entre les différentes actions du chasseur, comme le combat, la poursuite, le rechargement ou la pose de pièges. Cette organisation rend l'IA plus lisible et plus facile à faire évoluer.



7. Schéma du BT sans poids

Le système multijoueur ainsi que la synchronisation client/serveur sont également opérationnels, ce qui permet un comportement cohérent du chasseur pour tous les joueurs connectés.

Lien avec l'étude préalable

Les développements réalisés durant l'itération sont cohérents avec l'étude préalable. L'IA repose bien sur un Behaviour Tree. Même si certaines fonctionnalités restent à développer, la structure actuelle correspond à la vision initiale du projet

Itération 3

Objectifs

Les objectifs de cette itération sont :

- Des pièges fonctionnels

Le chasseur doit pouvoir placer un nombre limité de pièges. Si un lapin tombe sur un piège, le lapin perd de la vie et le chasseur récupère un piège.

- Un terrier

Un gros terrier relié par des entrées partout sur la carte. Un lapin peut entrer dans le terrier, dedans sa faim augmente, il peut aussi sortir où il veut.

- Le calcul dynamique des poids

Avoir une IA du chasseur plus humaine et plus réaliste avec des poids sur chaque action.

Avant l'Itération

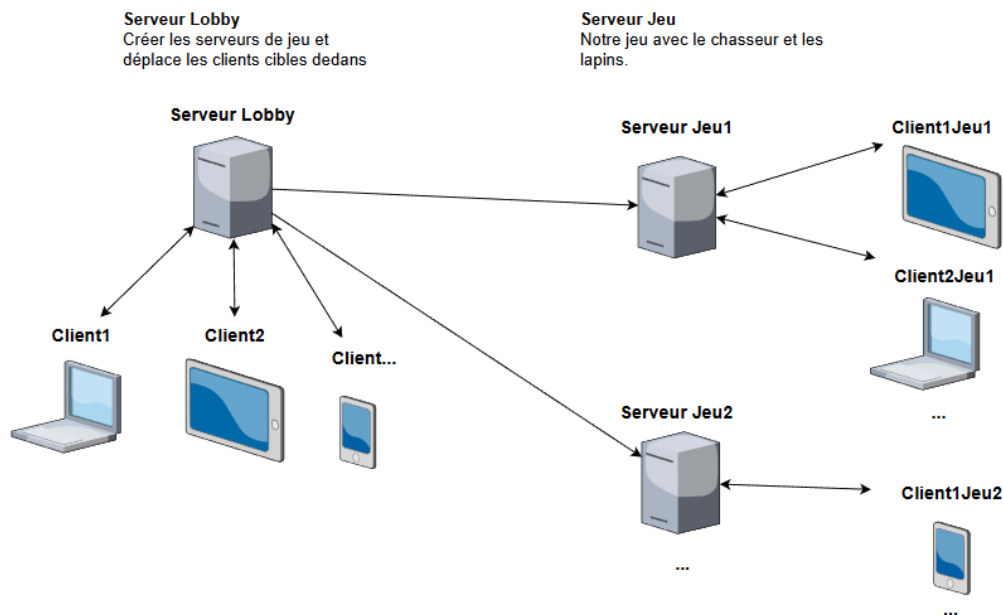
(Hugo Rochedreux)

A la fin de l'itération 2, nous avons remarqué que notre projet est un peu en retard par rapport aux autres, en grande partie due à l'absence de Noah pour raison médicale. C'est pourquoi, entre l'itération 2 et 3, j'ai décidé de rajouter des éléments plus graphiques qui prennent du temps.

J'ai donc ajouté :

- Modèle 3d chasseur
- Animations chasseur
- Lumière et ambiance
- Système de lobby

Les trois premiers points sont purement graphiques je ne vais pas les détailler, mais le système de lobby si.



8. Schéma de fonctionnement du lobby

Pour résumer l'image ci-dessus, quand un joueur se connecte au jeu, il est déplacé dans le serveur Lobby. Dedans, il peut créer un groupe, les autres peuvent le rejoindre et il peut commencer la partie. Quand la partie est lancée, le serveur Lobby va créer un serveur de jeu et déplacer tous les clients du groupe dedans.

Dans notre serveur de jeu, on va récupérer les informations telles que le nombre de joueurs, et on va attendre que tous les joueurs se connectent bien avant de lancer.

Bien sûr, si jamais un joueur met plus de 15 secondes à se connecter, la partie va commencer sans lui. Et si après les 15 secondes il se connecte il sera redirigé vers le serveur Lobby car la partie est en cours.

Déroulement

L'objectif principal de l'itération 3 est le calcul dynamique des poids (*cf Utility System*).

Pour ce faire nous avons commencé par énumérer chaque action et attribut de notre chasseur (*cf Formaliser le chasseur*) et ensuite réfléchi à chaque utility.

Nous avons donc créé les formules d'utility suivantes :

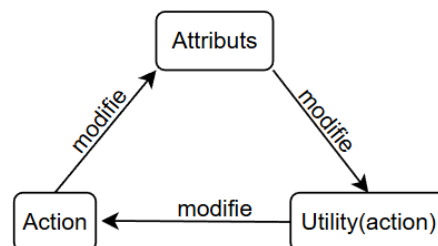
- Attaque = $A * (1-D) * (1-F)$
- Tir = $A * L * (B) * \max(0, 1 - \text{abs}(D-0.3)/0.6)$
- Placer Piège = $P * (1-F) * (1-D) * T$

- Recharge = $(1-B)*I$
- Ravitaillement = $(1-I)*(1-L)$
- Poursuite cible = $(1-F)*P*L$
- Poursuite Position = $P*(1-L)*(1-F)/2*O$
- Patrouille = $(1-L)*(1-\max(\text{Recharge}, \text{Ravitaillement}, \text{Poursuite Position}))$

Par la suite, nous avons commencé à modifier chaque attribut en fonction des actions du joueur.

Action	Fatigue	Patience	Agressivité	Munitions	Pièges
Attaque	+	-	+		
Tir	+	-	+	-	
Placer Pieu	-	+	-		-
Recharge	-	+	-	+	
Ravitaillement	-	+	-	+	
Poursuite cible	+	-	+		
Poursuite Position	+	-	+		
Patrouille	-	+	-		

Tout semble idéal, mais malheureusement nous n'avons pas appliqué les bonnes méthodes pour implanter l'utilité. Effectivement, nous avons essayé d'implanter tous les attributs et formules d'utilité en même temps. Cela a provoqué un effet de chaîne.



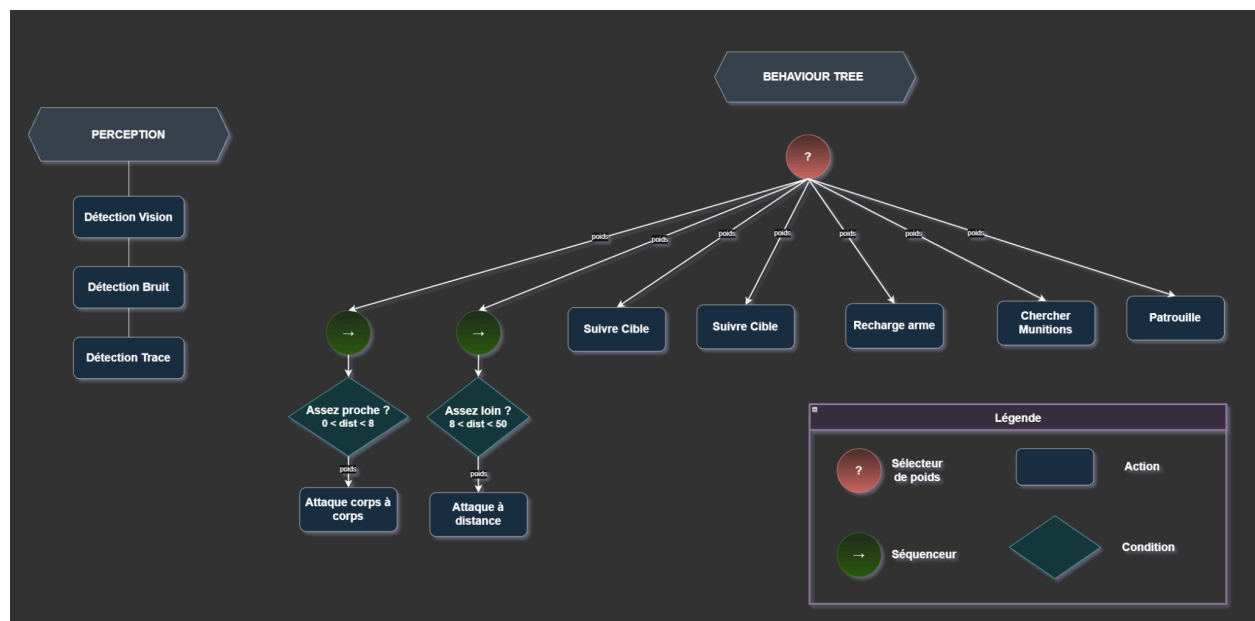
Pour comprendre le problème, prenons un exemple, l'action **Tir**. Cette action aurait comme formule $Utility(Tir) = Agressivité - DistanceJoueur$.

Eh bien, d'après notre tableau, l'action **Tir** augmente l'agressivité, or l'agressivité fait partie de notre formule d'utilité. Donc si la modification de l'agressivité est mal réglée, cela pourrait augmenter grandement l'utilité de l'action **Tir** et donc notre chasseur ne ferait que tirer ou inversement.

Nous avons donc mal étudié le problème et avons perdu beaucoup de temps à essayer de calibrer la modification des attributs. A la fin nous avons décidé de bloquer à des valeurs fixes certains attributs comme la fatigue, l'agressivité et la patience. Cette décision nous a permis d'obtenir une version du jeu jouable.

Pendant ce temps les pièges ont été faits.

Pour conclure nous avons voulu faire gros dès le départ par peur de ne pas faire assez et nous nous sommes retrouvés à l'inverse bloqués dans une complexité bien trop élevée et avons perdu beaucoup de temps à la régler.



9. Schéma du BT avec poids

Itération 4

Objectifs de l'itération

L'objectif principal de cette itération était d'améliorer le gameplay et l'expérience joueur, tout en poursuivant l'évolution de l'intelligence artificielle du chasseur.

Cette phase vise à enrichir les mécaniques de survie du lapin et à rendre les interactions avec le chasseur plus stratégiques.

Ajout de mécaniques buissons et terrier

Un système de buissons a été implémenté afin d'introduire une dimension de furtivité.

- Le lapin peut désormais se cacher dans un buisson
- Lorsqu'il est caché, il devient invisible pour le chasseur
- Cette action a un coût :
 - le lapin perd de la nourriture lorsqu'il entre et sort du buisson

Cette mécanique oblige le joueur à faire des choix : se cacher pour survivre ou économiser ses ressources.

Le système de terrier, prévu précédemment, a été ajouté et fonctionnel.

- Des entrées de terrier sont réparties sur la carte
- Le lapin peut entrer et ressortir par différentes sorties
- À l'intérieur du terrier :
 - le lapin est en sécurité
 - mais il perd 3 fois plus de faim

Caméra spectateur

Une nouvelle fonctionnalité a été ajoutée pour améliorer l'expérience après la mort du joueur :

- Lorsque le lapin meurt :
 - le joueur passe en mode spectateur
 - il peut observer les autres lapins

Utility actions

Le but de l'itération 4 était de se concentrer seulement sur quelques attributs :

- Balles chargeur
- Fatigue
- Distance du lapin: que de se concentrer seulement sur deux actions :
- Tirer
- Recharger

Les attributs sont normalisés entre 0 et 1.

Pour les balles dans le chargeur c'est plutôt simple, elles diminuent quand le chasseur tire et augmentent quand le chasseur recharge.

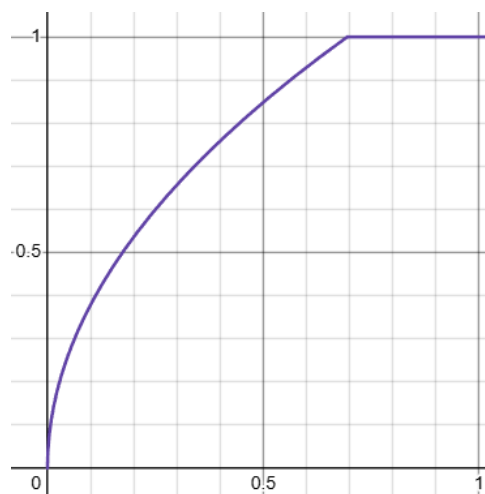
La distance lapin est aussi très simple, plus le lapin est proche, plus la valeur est faible, et si jamais le lapin est trop loin, la valeur est à 1.

La fatigue est le plus difficile à attribuer. On veut que certaines actions diminuent la valeur et certaines augmentent.

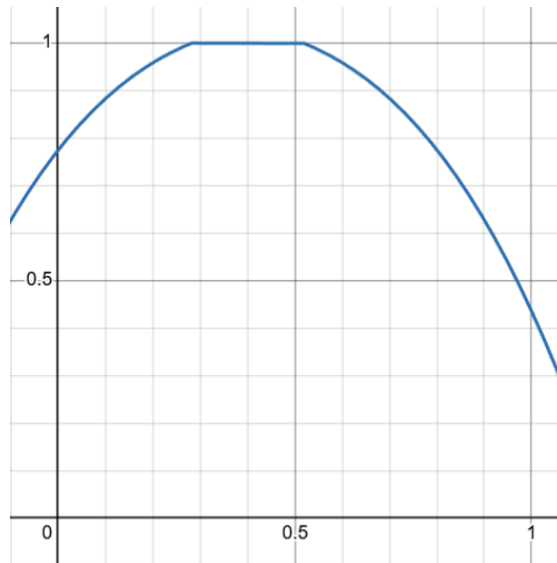
Pour tous les graphes l'abscisse est le paramètre Utility(abscisse) et la courbe représente l'utility de 0 à 1.

L'action Tir : **Utility(Tir)** = Utility(BalleChargeur)*max(0,Utility(Distance)-Utility(Fatigue))

Utility(BalleChargeur) = $\min(1, \sqrt{x} \cdot 1.2)$

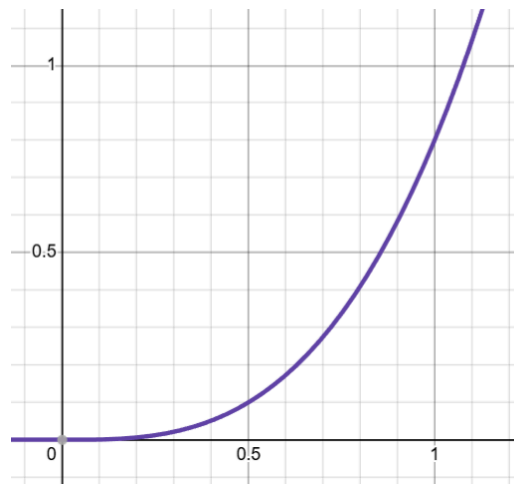


$$\text{Utility(InverseDistance)} = \max\left(0, \min\left(1, 1 - \left(\left|\text{abs}(x - 0.4) + 0.25\right|\right)^3\right)\right)$$



Pour expliquer, l'abscisse est la distance de 0 à 1, 1 étant très proche. On peut voir donc que quand le lapin est proche l'utilité augmente et l'utilité est au max entre 0.2 et 0.4 donc si le lapin est assez proche mais pas trop, l'utilité est à 1.

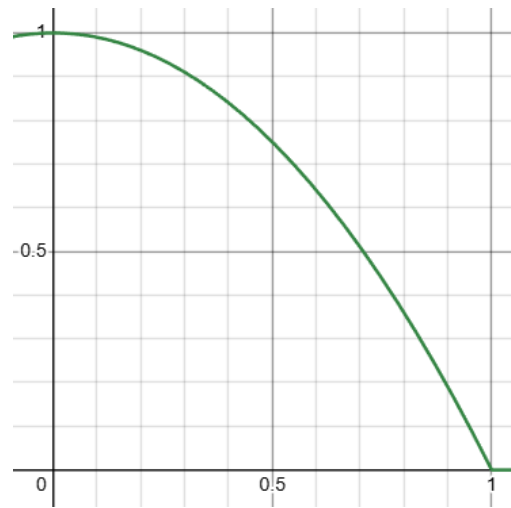
$$\text{Utility(Fatigue)} = \max\left(0, x^3 \cdot 0.8\right)$$



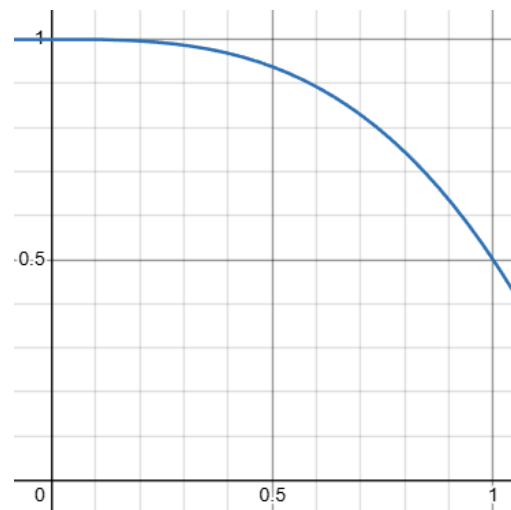
Voir sur [Desmos](https://www.desmos.com/calculator)

L'action Recharge : **Utility(Recharge)** = Utility(BalleChargeur)*Utility(DistanceLapin)

$$\text{Utility}(\text{BalleChargeur}) = \max\left(0, \min\left(1, 1 - x^2\right)\right)$$



$$\text{Utility}(\text{DistanceLapin}) = \max\left(0, \min\left(1, 1 - \left(\frac{x^3}{2}\right)\right)\right)$$



Itération 5

Objectifs de l'itération

Les objectifs de cette itération étaient de finaliser l'Utility AI du chasseur, d'améliorer significativement l'interface utilisateur côté client, et d'introduire un bot lapin pour enrichir l'expérience multijoueur.

État de la version

À la fin de cette itération, le jeu dispose d'une IA du chasseur complète et équilibrée, d'une interface utilisateur plus soignée, ainsi que d'un bot lapin fonctionnel capable d'évoluer de manière autonome dans l'environnement.

Finalisation des calculs d'Utility

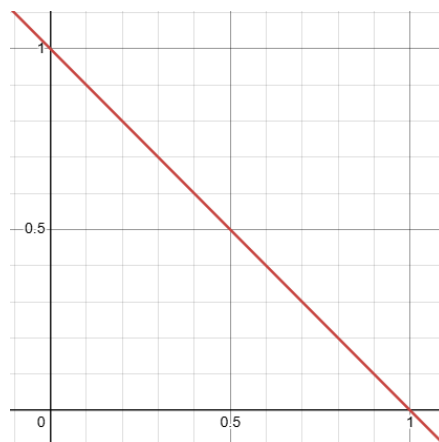
L'itération 5 avait pour priorité de finaliser l'ensemble des formules d'utility du chasseur, en étendant le travail initié lors de l'itération 4. L'objectif était ici de couvrir les actions restantes : Poursuite, Ravitaillement et Patrouille.

Etant donné un manque de personnel et de temps, les utility ci-dessous sont beaucoup plus simple que celle faite à l'itération 4.

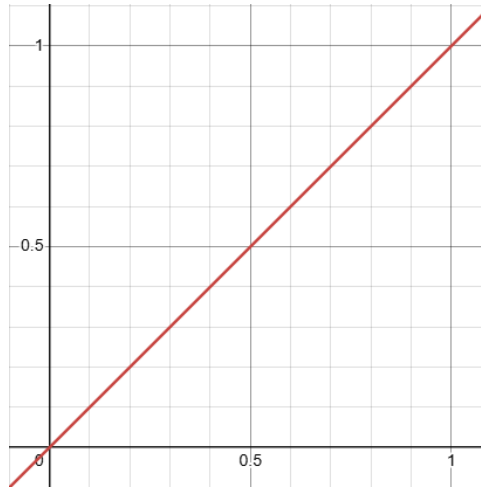
Pour tous les graphes l'abscisse est le paramètre Utility(abscisse) et la courbe représente l'utility de 0 à 1.

L'action Poursuite : **Utility(Poursuite)** = Utility(FatigueInverse) * Utility(LigneDeVue) * Utility(Patience) * Utility(BallesReserve)

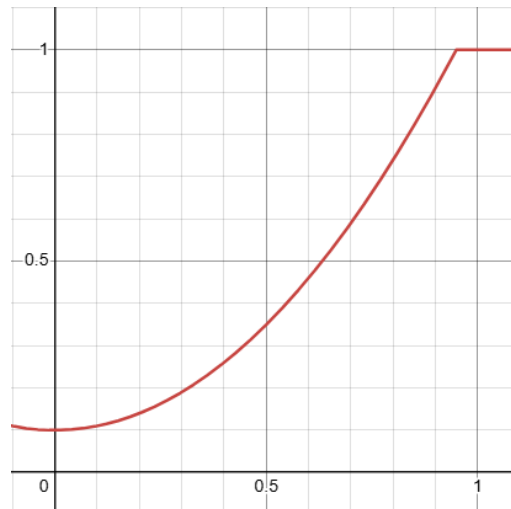
Utility(FatigueInverse) = $1 - x$



Utility(LigneDeVue) et Utility(Patience) = x



Utility(BallesReserve) = $\min(0.1 + x^2, 1)$

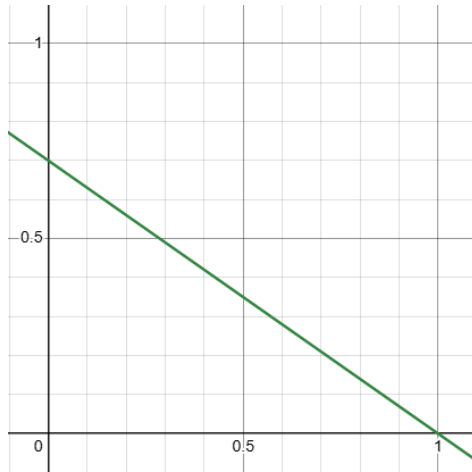


La poursuite est maximale quand :

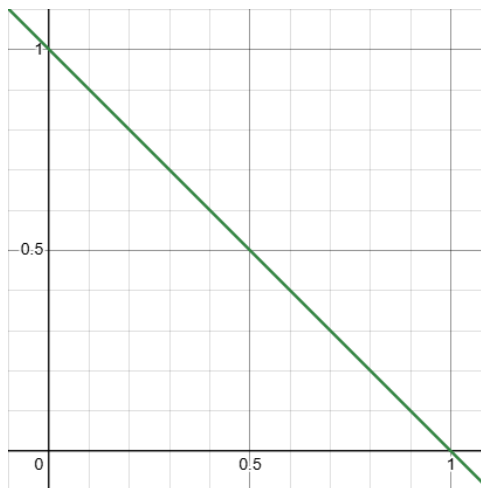
- le chasseur est en forme,
- il voit le lapin,
- il est patient,
- et il a beaucoup de munitions.

L'action Poursuite : **Utility(GetMunitions)** = Utility(ManqueBalles) * Utility(PasDeVision)
* Utility(FatigueModérée)

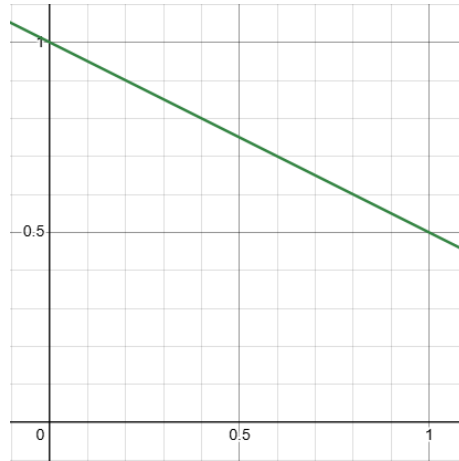
$$\text{Utility(ManqueBalles)} = (1 - x) \cdot 0.7$$



$$\text{Utility(PasDeVision)} = 1 - x$$



$$\text{Utility}(\text{FatigueModérée}) = 1 - \frac{x}{2}$$



La recherche de munitions est maximal quand :

- le chasseur est presque à court,
- le chasseur ne voit pas le lapin,
- et le chasseur a encore assez d'énergie.

L'action Exploration : **Utility(Exploration)** = $\text{Utility}(\text{PasDeVision}) \times (1 - \max(\text{Utility}(\text{AutresActions})))$

L'Utility(PasDeVision) est la même que celle au-dessus. On a :

max(Utility(AutresActions)) = max(
 Utility(Recharge),
 Utility(GetMunitions),
 Utility(Poursuite)
)

Si une autre action est très utile → exploration devient inutile

Si aucune action n'est intéressante → exploration augmente

Bot lapin

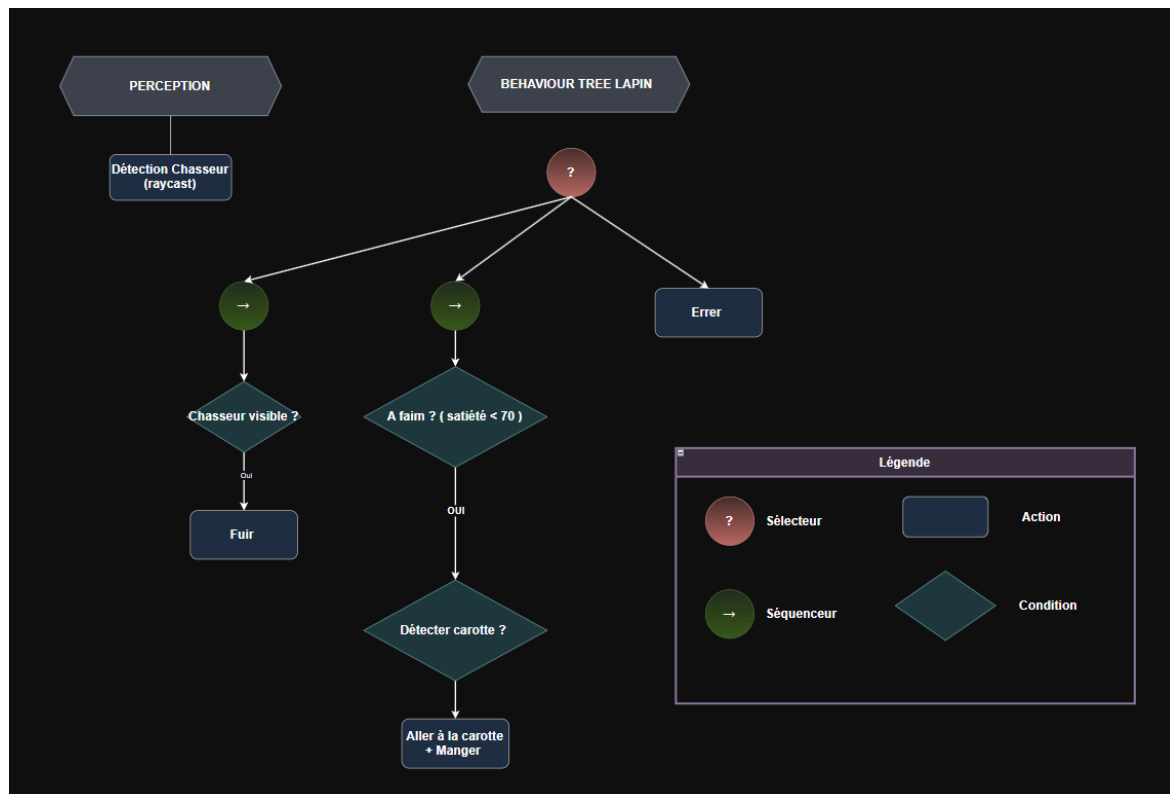
Un bot lapin a été intégré pour permettre au chasseur d'avoir plusieurs cibles actives même en l'absence de joueurs humains supplémentaires. Ce bot adopte un comportement simple : il se déplace aléatoirement sur la carte, réagit à la proximité du chasseur en fuyant, et utilise les mécaniques existantes (buissons, terriers).

Ce lapin bot repose sur une machine à états finis (FSM) légère, distincte du Behaviour Tree du chasseur, afin de rester simple à maintenir.

L'ajout de ce bot remplit également un rôle de test : il permet d'observer le comportement du chasseur face à plusieurs cibles et de vérifier la robustesse du Weighted Selector dans des situations à cibles multiples.

L'interface utilisateur a été enrichie et rendue plus lisible :

- Ajout d'une **minimap** affichant la position du lapin sur la carte, sans révéler celle du chasseur.
- Refonte des **jauges** de vie et de faim pour un rendu plus lisible et plus cohérent avec l'ambiance du jeu.
- Ajout d'une **jauge de temps** permettant au joueur de suivre la durée de la partie en cours.
- Amélioration visuelle des **bordures de map**, rendues plus esthétiques et mieux intégrées à l'environnement.
- Amélioration générale du **lobby** : meilleure lisibilité de la liste des joueurs et retour visuel lors du lancement d'une partie.
- Ajout d'un **bouton de saut** dédié à la version mobile, afin de garantir une expérience jouable sur tous les supports.
- Implémentation de la **détection des bruits** produits par les joueurs et les bots, permettant au chasseur de réagir aux sons de l'environnement.



10.

Schéma du BT des lapins bots

Difficultés rencontrées

La principale difficulté de cette itération a été la gestion du bot lapin en contexte multijoueur. La synchronisation de son état entre le serveur et les clients a nécessité des ajustements pour éviter des désynchronisations visibles. Par ailleurs, l'équilibrage final de l'Utility a demandé de nombreux tests en conditions réelles pour obtenir un chasseur ni trop passif ni trop agressif.

Itération 6

Objectifs de l'itération

Les objectifs de cette dernière itération étaient de stabiliser et d'équilibrer le jeu dans son ensemble, de corriger les derniers bugs identifiés, d'ajuster les paramètres du chasseur pour une expérience de jeu optimale, et de préparer les livrables finaux : rapport, présentation et soutenance.

État de la version

À l'issue de cette itération, The Silent Hunt est dans sa version finale. Le jeu est stable, jouable et équilibré. Les dernières corrections apportées garantissent une expérience cohérente aussi bien pour un joueur seul qu'en multijoueur.

Correction des bugs du lapin

Plusieurs bugs identifiés au cours des itérations précédentes ont été résolus lors de cette phase finale. Les principaux problèmes concernaient le comportement du lapin dans certaines situations limites : collisions incorrectes, et certains problèmes d'animations. Ces corrections ont permis d'assurer une expérience de jeu fluide et sans accrocs pour le joueur.

Ajustement des paramètres du chasseur

Un travail important d'équilibrage a été réalisé sur les paramètres du chasseur afin de rendre la partie la plus pertinente et agréable possible. Les valeurs clés comme la vitesse de déplacement, la portée de vision, la cadence de tir et les seuils d'utilité ont été ajustés après plusieurs sessions de tests.

L'objectif était de trouver un équilibre entre un chasseur trop facilement évitable et un chasseur imbattable.

Préparation des livrables finaux

Cette itération a également été consacrée à la préparation de l'ensemble des livrables attendus en fin de projet :

- Rédaction et finalisation du **rapport final**
- Préparation de la **soutenance finale**
- Conception d'une **présentation pour des étudiants de première année**.

7. Répartition du travail

Hugo	Noah	Emma
Structure arbre de comportement	Animation du chasseur	Système de vision du chasseur
Multijoueur / Système lobby	Début du système de buissons	Attaque à distance
Synchronisation données client/serveur	Interface utilisateur	Gestion des munitions
Gestion de mort du lapin	Actions du joueur	Ravitaillement des munitions
Apparition des carottes	Début de la map	Système de pièges
Sons du lapin et des objets	Terriers	Intégration dans le BT
Modèle chasseur	Calculs de poids	Pathfinding
Mouvement lapin / Caméra		Tests
Calculs de poids		Lapin Bot
TOUT l'ux/ui		

Planning de déroulement du projet durant les 6 itérations

Itération 1	Itération 2	Itération 3
Mise en place des bases du projet	Amélioration du Behavior tree	Début de l'Utility AI
Création du lapin jouable	Détection, poursuite et combat	Finalisation des pièges
Première version du chasseur	Gestion des munitions	Implémentation du terrier
Carte de test	Début des pièges	Amélioration des animations
Prise en main de Lua et de Roblox	Multijoueur	Tests et équilibrage

Itération 4	Itération 5	Itération 6
Calcul des utility	Lapin Bot terminé	Rédaction des livrables
Terriers	Finitions UX / UI	Amélioration de l'IA
Buissons	Gestion multijoueur	Présentation aux BUT 1
Tests	Finitions calculs utility	
Nouvelles fonctionnalités UX	Polissage et derniers débogages	
Début Lapin Bot		

Conclusion

Au terme de ces six itérations, *The Silent Hunt* a réellement pris forme. Le projet est passé d'un simple prototype à un jeu complet, jouable, et doté d'une IA de chasseur crédible. Chaque itération a apporté ses améliorations : d'abord les bases du gameplay et du multijoueur, puis l'amélioration du Behaviour Tree, l'ajout des pièges, du terrier, et l'intégration d'un Utility System capable d'adapter dynamiquement les décisions du chasseur.

Le développement n'a pas été linéaire. Entre la prise en main de Roblox, la complexité du modèle client/serveur, l'équilibrage de l'IA et les imprévus d'organisation, nous avons dû revoir nos priorités plusieurs fois. Mais ces difficultés nous ont forcés à structurer notre travail, à mieux communiquer et à nous concentrer sur les fonctionnalités essentielles. Le résultat est une IA plus stable, un gameplay cohérent et une architecture suffisamment solide pour accueillir les futures améliorations.

Annexe

Lancer le jeu

Pour pouvoir lancer le jeu vous pouvez aller sur cette [page](#) roblox. Afin de jouer il vous faut un compte roblox. Vous pouvez vous créer un compte sur ce [lien](#).

Une fois le jeu lancé vous aurez cette page.



Vous pourrez aller voir le tutoriel ou alors jouer au jeu.

Dans le menu jouer vous pouvez rejoindre les lobby existant ou créer un lobby.



Dans la création de lobby, vous pouvez sélectionner un nombre de bot et un nombre de joueurs maximum.



Dans le lobby vous voyez les joueurs qui sont avec vous et vous pouvez les exclure du lobby. Vous pouvez lancer la partie.



Une fois en jeu vous pouvez vous déplacer avec ZQSD, interagir avec votre environnement (carrot, terrier, ..) avec E.
Enfin la touche I fait apparaître un panneau à gauche avec les informations du chasseurs.



Installation du jeu

Pour installer le projet et pouvoir le modifier il y a deux solutions.

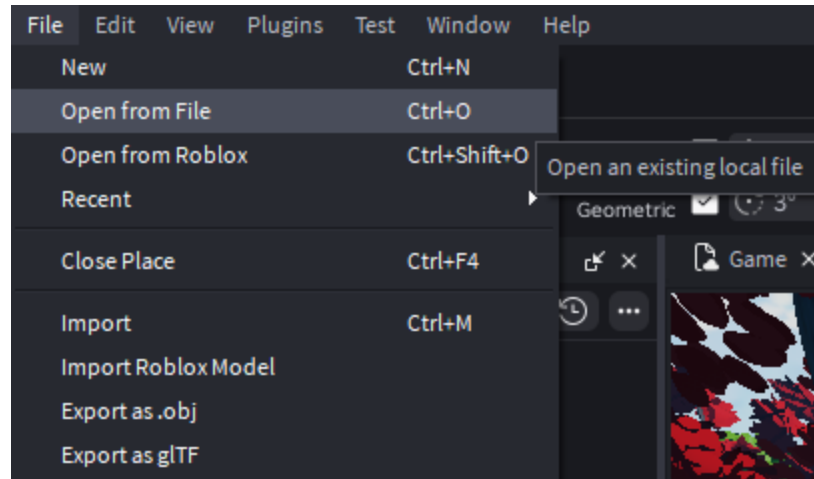
Premièrement vous pouvez tout simplement voir les scripts dans un IDE liée à notre github mais vous ne pourrez pas les lancer.

Lié le github à VsCode et puis VsCode à ROJO et ROJO à Roblox Studio est un processus assez complexe.

C'est pourquoi vous pouvez directement installer et lancer [Roblox Studio](#). Ainsi vous pourrez modifier les scripts directement dans le moteur de jeu et lancer le jeu directement en local.

Une fois Roblox Studio installé dans github il suffit de récupérer dans document le fichier Save.rbxl.

Puis dans Roblox Studio ouvrez le fichier Save.rbxl



Une fois le fichier ouvert vous pouvez lancer en local le jeu sans passer par le lobby

